



**POLITECNICO**  
MILANO 1863

**Intelligent Job Management – Jan Wangrat**

# **Intelligent Job Management**

## **Final Report**

**Author:** Jan Wangrat

**Supervisor:** Prof. Danilo Ardagna, Ph.D. Candidate Federica Filippini

**Date:** May 18, 2026

---

|                     |                                                          |
|---------------------|----------------------------------------------------------|
| <b>Deliverable:</b> | Final Report                                             |
| <b>Title:</b>       | Intelligent Job Management                               |
| <b>Author:</b>      | Jan Wangrat                                              |
| <b>Supervisor:</b>  | Prof. Danilo Ardagna, Ph.D. Candidate Federica Filippini |
| <b>Version:</b>     | 1.0                                                      |
| <b>Date:</b>        | May 18, 2026                                             |

---

## Contents

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Table of Contents</b>                           | <b>3</b>  |
| <b>List of Figures</b>                             | <b>4</b>  |
| <b>List of Tables</b>                              | <b>4</b>  |
| <b>1 Introduction</b>                              | <b>6</b>  |
| <b>2 System Architecture</b>                       | <b>7</b>  |
| 2.1 Deployment topology                            | 7         |
| 2.2 Scheduling pipeline                            | 9         |
| 2.3 State machine                                  | 13        |
| 2.4 Concurrency invariants                         | 13        |
| 2.5 Cross-node checkpoints and image variants      | 14        |
| <b>3 Configuration &amp; Job Submission</b>        | <b>15</b> |
| 3.1 Cluster Configuration                          | 15        |
| 3.1.1 Node Configuration (nodes_config.json)       | 15        |
| 3.1.2 GPU Energy Costs (gpu_energy_costs.json)     | 16        |
| 3.2 Job Submission                                 | 16        |
| 3.2.1 Request Body                                 | 16        |
| 3.2.2 Response (201 Created)                       | 17        |
| 3.3 Full API Overview                              | 17        |
| 3.4 Runtime environment                            | 18        |
| <b>4 End-to-End Scenarios</b>                      | <b>19</b> |
| 4.1 Objective and cost model                       | 19        |
| 4.2 Available job types                            | 19        |
| 4.3 Scenario 1 — single type                       | 20        |
| 4.4 Scenario 2 — two types, forced 2-GPU placement | 23        |
| 4.5 Reproducibility                                | 24        |
| <b>References</b>                                  | <b>25</b> |

## List of Figures

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Distributed prototype: SPA + API + optimizer on the operator’s machine, workers + Docker on each GPU node, PostgreSQL on a GPU node, all glued together over SSH-tunneled HTTP. Checkpoints written by one node are visible from the other via a shared rclone-over-SFTP mount. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 8  |
| 2 | Submission and scheduling. The API only writes the row and notifies the scheduler; placement is decided by the next scheduling pass. The optimizer call is made outside the schedule lock so its 30 s HTTP timeout cannot block other passes; stale optimizer suggestions are safely filtered by Phase 1b’s conditional update. Slot semaphores ensure that no more containers than GPUs are dispatched to any worker. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 10 |
| 3 | Profiling cycle. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 11 |
| 4 | Optimizer cycle. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 12 |
| 5 | Standard run execution. The worker performs an atomic conditional UPDATE . . . WHERE status = ANY(RUNNABLE_STATUSES) so concurrent stops / duplicate dispatches cannot race a row into RUNNING. On exit the worker fires both <code>ijm_slot_freed</code> (which releases the API’s NodeSlots permit) and <code>ijm_schedule</code> (which wakes the scheduler) in the same transaction. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 13 |
| 6 | Scenario 1 timeline. <i>Bar widths are worker container lifetimes</i> , not measurement durations: profile measurement for a cell completes once the worker has emitted enough post-warmup epoch timestamps (typically within a handful of epochs), but the runtime does not exit on measurement completion — the container keeps training standard epochs until the scheduler either claims a new config (J2’s transition from P600×1 to P600×2 at $t=4.17$ ) or an external event stops it (URGENT-driven preempt or user-stop). Profile sweep is carried by JOB1 (polimi-gpu, hatched blue: A40×1 then A40×2) and JOB2 (matemagician, hatched red: P600×1 then P600×2 <sup>×2</sup> ); together they cover all four cells. <code>cnn_big</code> on P600 is $\sim 8\times$ slower per epoch than on A40, which is why JOB2’s profile bars are much wider than JOB1’s — the per-epoch profile measurements end up being 47 s and 28 s respectively (see Table 10). JOB3 and JOB4 find the cache populated by the time they reach the optimiser and skip the sweep. At $t=5.38$ min URGENT arrives with a 10-min deadline and 200 epochs — unmeetable on every config, so the optimiser picks the fastest (A40×1) and auto-preempts the lowest-priority polimi-gpu resident (JOB4). URGENT dispatches on A40×1 once JOB4’s container has drained ( $t=5.85$ ). At $t=7.49$ the operator user-stops JOB2 (still in PROFILING — progress 1/50; the user-stop is sticky, JOB2 stays in PREEMPTED), freeing matemagician’s P600. JOB4 then <i>resumes on matemagician</i> from the legacy-format checkpoint — <i>cross-node</i> relative to its polimi-gpu origin — and runs to completion on P600×1. URGENT continues on A40×1 and SUCCEEDS at $t\approx 40.6$ min (off-chart); JOB3 and JOB4 finish on matemagician at $t\approx 45$ min. Slot use never exceeds 2 jobs/node at any instant. . . . . | 21 |
| 7 | Scenario 2 timeline (snapshot ground truth, 0.5 s poll). Profile sweep covers all 8 cells across two types. PIN-B’s huge hatched bar (0.12–4.13 min) is cold <code>cnn_big</code> P600×1 profile: $47.7\text{ s/epoch} \times 3\text{ epochs} + \text{container start}$ . Steady state at $\sim 8$ min: both A40 slots hold the <code>cnn_big</code> pins, both P600 slots hold the <code>lstm-small</code> patients. At $t=8.22$ URGENT arrives and the optimiser places it on <i>matemagician</i> × <i>QuadroP600</i> ×2 (taller red bar), evicting <i>both</i> <code>lstm-small</code> patients in a single cascade — the central result. URGENT runs 27 epochs on P600×2 from 8.22 to 29.91, then migrates to polimi A40×1 (freed when PIN-A and PIN-B finished at 16.48 and 22.02) and completes the final 13 epochs there (SUCCEEDED 32.65). At 29.91, both <code>lstm-small</code> patients resume on the freed P600 slots from their checkpoints and finish $\sim t=60$ min (off-chart). Slot use never exceeds 2 jobs/node at any instant. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 23 |

## List of Tables

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Node configuration fields. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 15 |
| 2  | Job creation request fields. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 16 |
| 3  | API endpoints. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 17 |
| 4  | Key environment variables. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 18 |
| 5  | Bundle hourly rates and per-priority tardiness weights. Two-GPU bundles include a small co-location discount. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 19 |
| 6  | Training containers available in runtime/. All share the same checkpoint contract (legacy pre-1.6 torch serialization, loaded with <code>strict=False</code> ), so any job can resume across nodes regardless of the destination’s torch version. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 19 |
| 7  | Steady-state per-epoch wall-clock from <code>profiling_results.duration_seconds</code> (mean across epochs 2–3, excluding the warmup). Bold cells mark 2-GPU bundles that beat their 1-GPU sibling. <code>cnn_big</code> has a substantial 2-GPU win on P600 (1.69×); on A40 the 2-GPU configuration is essentially tied with 1-GPU (5.146 vs 5.144 s/ep), so adding the second A40 doubles the hourly rate for no speed-up and is strictly dominated by A40×1 in the cost model. <code>efficientnet</code> ’s legacy image uses <code>torch.nn.SiLU</code> which is unavailable on the <code>matemagician</code> PyTorch 1.5.1 image, so its P600 cells are unmeasurable. Dashes mark types not exercised by the scenarios in this report. . . . . | 20 |
| 8  | GPUcost per epoch = (per-epoch [s]) × (hourly rate from Table 5) / 3600. Multiply by <code>epochsTotal</code> to get the optimiser’s energy term for a given plan. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 20 |
| 9  | Scenario 1 submissions. Two tight prio-4 patients (the “cheap A40×1 candidates”), two slack low-prio patients (“cheap P600×1 candidates”), one URGENT submitted after the patients have settled into standard runs. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 20 |
| 10 | Per-bundle data the optimiser sees for <code>cnn_big</code> after S1’s profile sweep. Per-epoch is the steady-state mean from <code>profiling_results</code> (warmup epoch excluded); 50-ep / 200-ep costs use Table 5 hourly rates. P600×1 strictly minimises cost for any deadline it can meet; for jobs whose deadline rules out the 39-min P600×1 wall, A40×1 is the cheapest deadline-meeting option. . . . .                                                                                                                                                                                                                                                                                                                                  | 21 |
| 11 | Optimiser plan comparison for URGENT. <code>ExecTime</code> = 200× per-epoch from Table 10; tardiness measured against URGENT’s 10-min deadline at submission; <code>TardCost</code> uses the priority-5 weight $w_5=0.000444$ \$/s (Table 5). A40×1 wins because its time is essentially tied with A40×2 but the energy cost is roughly half, and it only needs to evict <i>one</i> <code>polimi</code> resident (the lowest-priority one, JOB4) instead of both. P600×2 sits at second cheapest on paper but its profile was still incomplete at submission time, so the heuristic could not commit to it. . . . .                                                                                                                                | 22 |
| 12 | Scenario 2 submissions. PIN-A/B are tight-deadline <code>cnn_big</code> jobs that the optimiser is forced to place on <code>polimi-gpu</code> A40×1 (P600×1 would be 68 min runtime vs their 10-min deadline). P-LSTM-1/2 are slack <code>lstm-small</code> jobs that land on <code>matemagician</code> P600×1. URGENT arrives once steady state holds; its tight deadline admits P600×2 (20 min) but not P600×1 (34 min), and the A40 plans are blocked by the priority-5 tight-deadline PIN victims. . . . .                                                                                                                                                                                                                                      | 23 |
| 13 | Optimiser plan comparison for URGENT. <code>ExecTime</code> is 40 × per-epoch from Table 7; tardiness measured against URGENT’s deadline (submit +22 min); <code>TardCost</code> uses the priority-5 weight from Table 5. † A40 plans appear free of URGENT’s own tardiness, but their <i>cascade cost</i> dominates: each priority-5 pin victim re-dispatched onto P600×1 becomes 58 min tardy on its own 10-min deadline ( $\approx \$1.55/\text{victim}$ ). P600×2 evicts only the slack <code>lstm-small</code> s (+8 h deadline) so its cascade is $\approx \$0$ . Captured run confirms the chosen plan. . . . .                                                                                                                              | 24 |

# **1 Introduction**

## 2 System Architecture

### 2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* (which also hosts the *ProfilingScheduler*, *JobDispatcher*, *NodeSlots* semaphores and the optimizer client) runs on the operator's machine. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes. Communication is plain HTTP over SSH tunnels, so the API reaches each worker at a stable `http://localhost:800x` alias regardless of where it actually sits. An optional GPUspb *optimizer* HTTP service is reached from the API.

## Current Prototype Architecture (distributed)

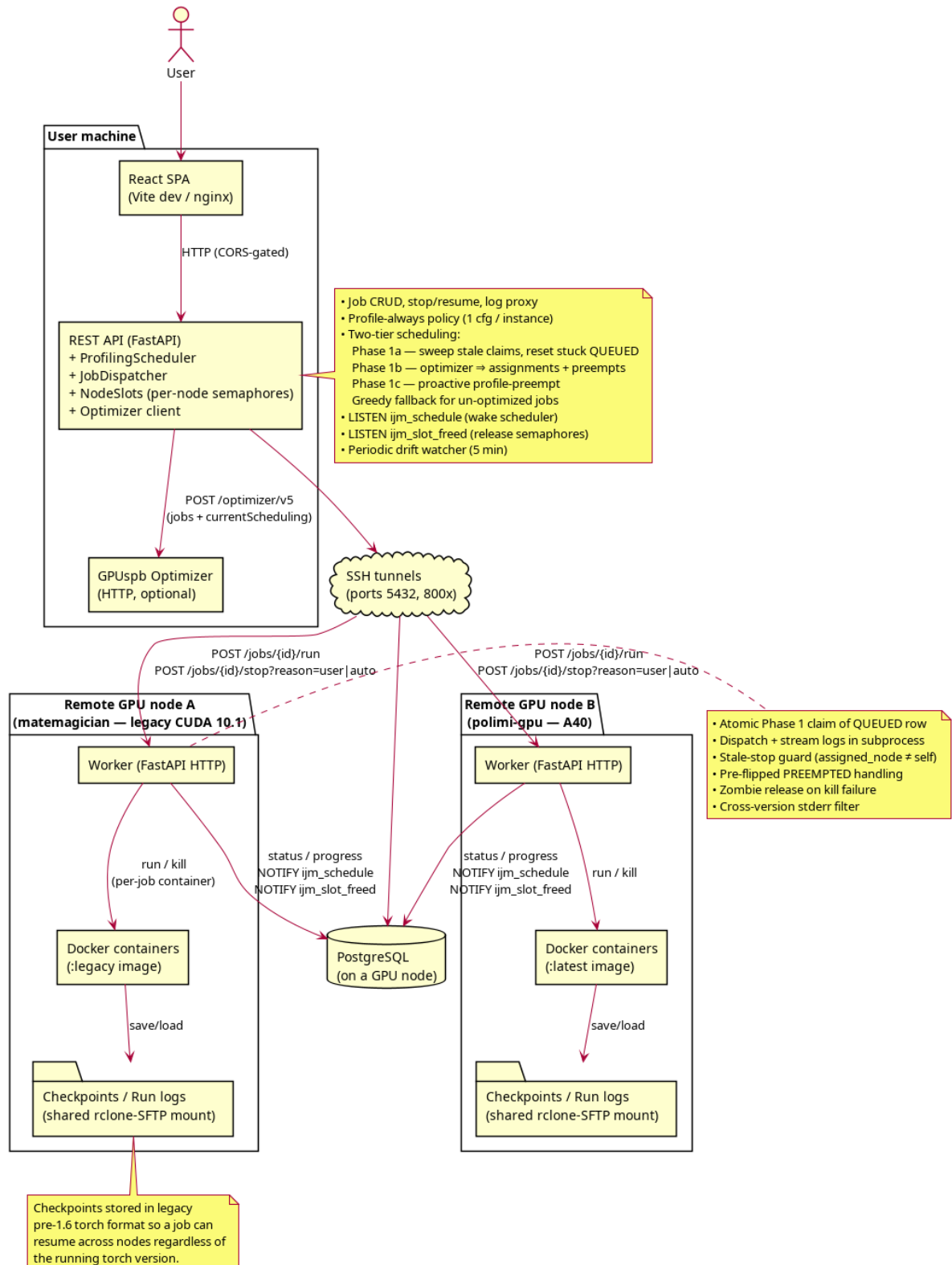


Figure 1: Distributed prototype: SPA + API + optimizer on the operator’s machine, workers + Docker on each GPU node, PostgreSQL on a GPU node, all glued together over SSH-tunneled HTTP. Checkpoints written by one node are visible from the other via a shared rclone-over-SFTP mount.

Single-process docker compose on the operator’s machine is supported for development but is not



the deployment target: any tradeoff between local-dev simplicity and distributed correctness resolves toward the distributed topology.

## 2.2 Scheduling pipeline

A single scheduling pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision. It is invoked whenever a job is submitted, a profiling run finishes, a slot is freed, or every 60 s by a watchdog. Notices are coalesced through a single `asyncio.Event` and rate-limited to at most one pass every five seconds so the cluster has time to apply the previous plan before the next one is computed.

The pass walks four phases under a single `schedule_lock`:

**Phase 0** Sweep stale profile claims (rows in `profiling_results` with `duration_seconds` IS NULL whose owning instance no longer exists or was repurposed) and reset stuck QUEUED rows whose `assigned_node` was set  $\geq 30$  s ago without the worker advancing them.

**Phase 1b** Call the optimizer (outside the lock for its 30 s HTTP timeout) and apply its assignments via conditional UPDATE ...WHERE `assigned_node` IS NULL; collect its preempt list.

**Greedy fallback** For QUEUED rows the optimizer did not place (typically rows still owing a profile run), the `ProfilingScheduler` picks one profile config or a profiled standard slot.

**Phase 1c** If a row still has no slot and still owes a profile, find the cheapest set of low-priority RUNNING jobs whose eviction would free enough GPUs for the next unprofiled config (the *profile-always* policy).

Outside the lock the pass spawns one task per preempt and one per dispatch. Each dispatch goes through `JobDispatcher.dispatch_with_slot`, which acquires the destination's `NodeSlots` permit before sending POST `/jobs/{id}/run` to the right worker. Permits are released when the worker emits a NOTIFY `ijm_slot_freed` carrying the freed (`node_id`, `n_gpus`) pair.

Figure 2 shows the submission path, Figure 3 the cross-instance profile cycle, Figure 4 an optimizer-driven preempt+migrate round, and Figure 5 the standard run.

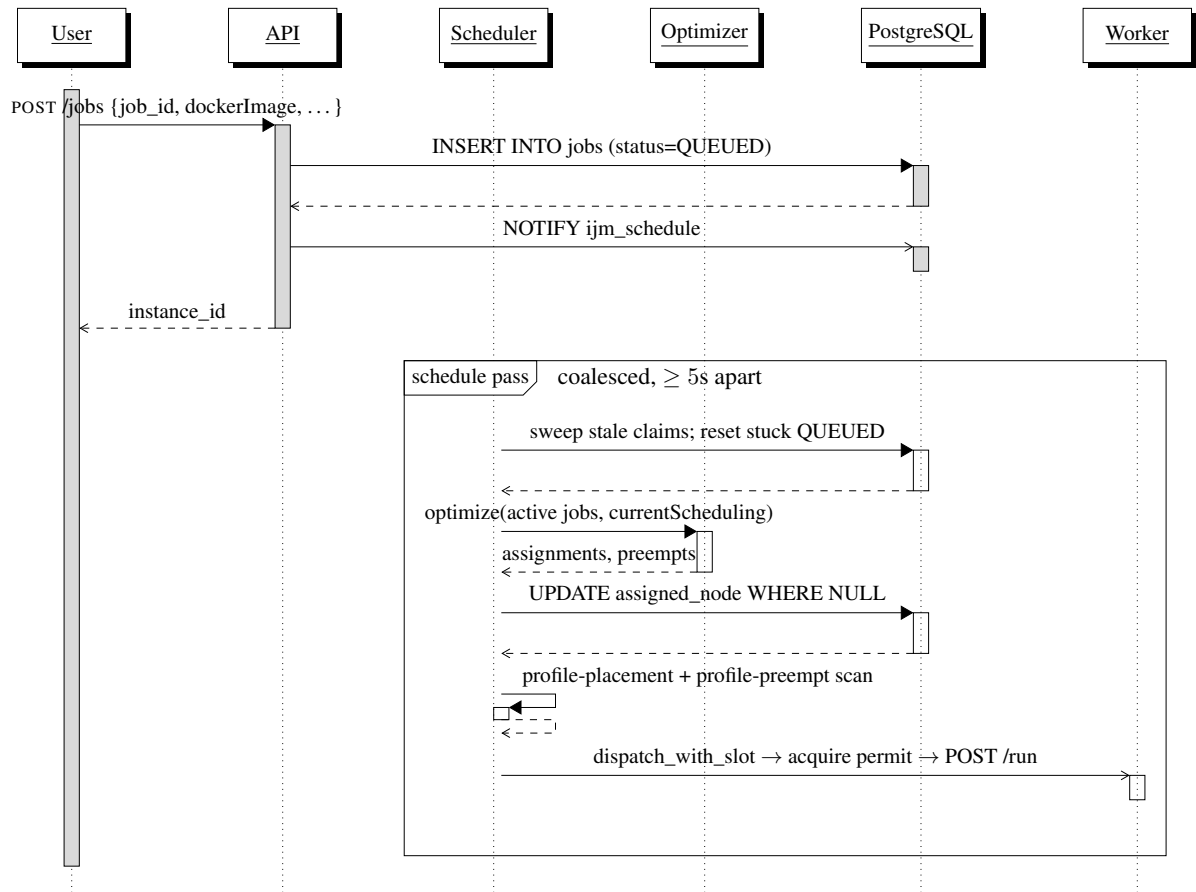


Figure 2: Submission and scheduling. The API only writes the row and notifies the scheduler; placement is decided by the next scheduling pass. The optimizer call is made outside the schedule lock so its 30 s HTTP timeout cannot block other passes; stale optimizer suggestions are safely filtered by Phase 1b’s conditional update. Slot semaphores ensure that no more containers than GPUs are dispatched to any worker.

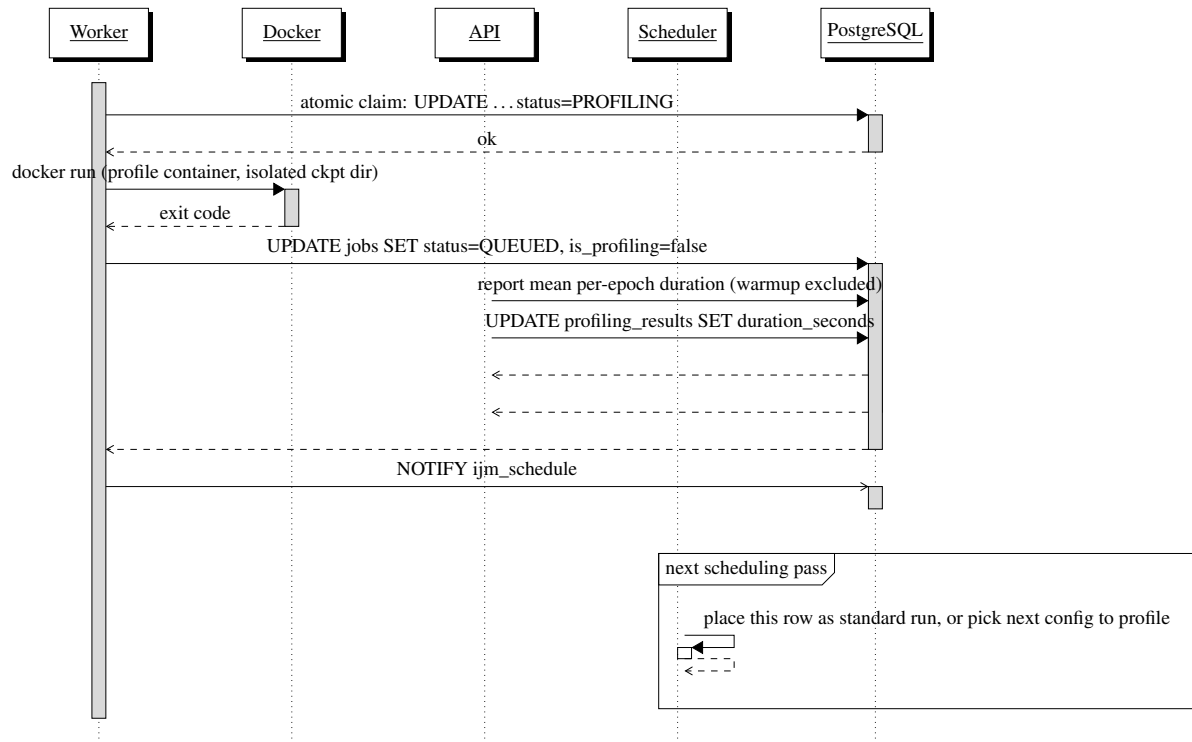


Figure 3: Profiling cycle. Profiling results are keyed by the (job-type, gpu-config) pair and shared across submissions of the same type. Each instance contributes one profile run by default (PROFILING\_CONFIGS\_PER\_JOB); the per-instance claim row guards against duplicate measurements. Mean per-epoch time is computed from monotonic timestamps with the first (warmup) epoch excluded.

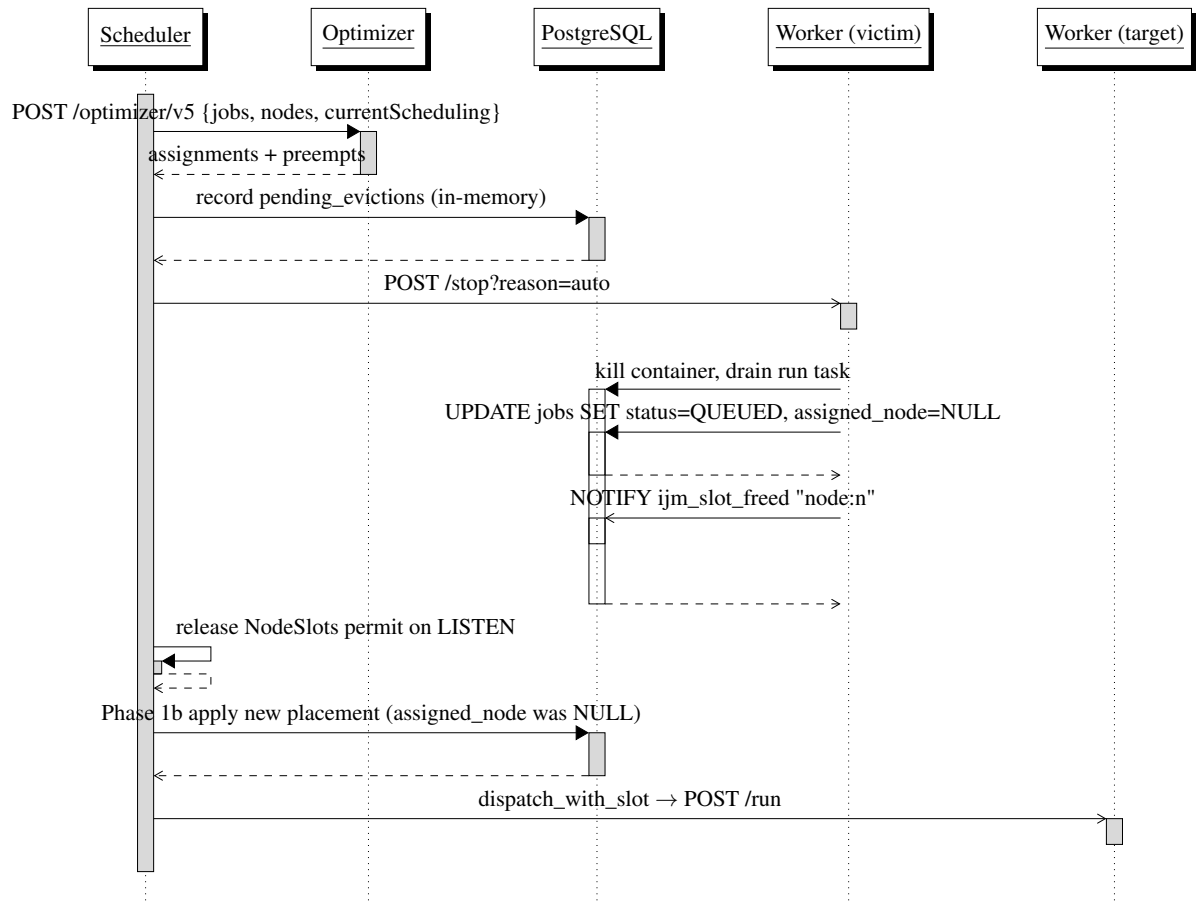


Figure 4: Optimizer-driven preempt and migrate. The API trusts GPUspb’s plan: any job in `currentScheduling` that the optimizer drops or moves to a different slot is preempted with `reason=auto`, which clears its assignment so Phase 1b can re-apply the new placement. An in-memory pending-evictions map makes freshly-evicted slots visible to the scheduler before the worker’s slot-freed commit lands, so multi-victim profile preempts do not deadlock.

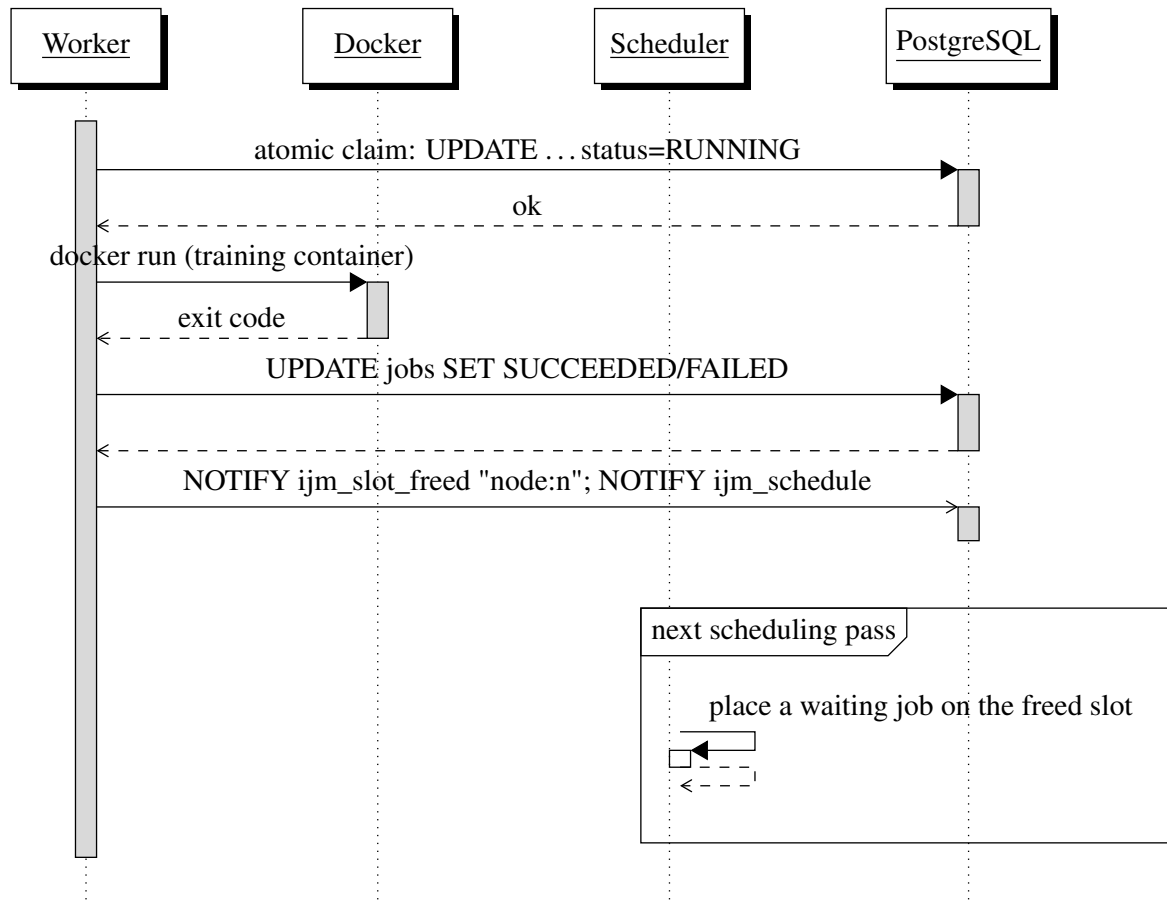


Figure 5: Standard run execution. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops / duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires both `ijm_slot_freed` (which releases the API's NodeSlots permit) and `ijm_schedule` (which wakes the scheduler) in the same transaction.

## 2.3 State machine

A job moves through the following states. All transitions are driven by atomic conditional `UPDATES` so concurrent stop / dispatch / completion paths cannot clobber each other's work.

| From               | To                  | Trigger                                                                                                        |
|--------------------|---------------------|----------------------------------------------------------------------------------------------------------------|
| QUEUED             | PROFILING / RUNNING | Worker Phase 1 atomic claim                                                                                    |
| PROFILING          | QUEUED              | Profile run finished; <code>is_profiling_run</code> flips to false                                             |
| RUNNING            | SUCCEEDED / FAILED  | Container exit                                                                                                 |
| RUNNING            | PREEMPTED           | User POST <code>/jobs/{id}/stop?reason=user</code>                                                             |
| RUNNING            | QUEUED              | Auto-preempt <code>reason=auto</code> : clears <code>assigned_node</code> so the next pass can place elsewhere |
| PREEMPTED / FAILED | QUEUED              | POST <code>/jobs/{id}/resume</code>                                                                            |

Two distinct stop reasons separate *user-initiated* preemption (sticky `PREEMPTED`, the user must explicitly resume) from *scheduler-initiated* preemption (`QUEUED` with cleared assignment, picked up automatically on the next scheduling pass). The worker enforces this distinction and is also responsible for ignoring stale `/stop` calls aimed at a row that has since been re-assigned to a different node.

## 2.4 Concurrency invariants

The control plane relies on three invariants, each defended in code:

- **No GPU oversubscription.** Every dispatch goes through `NodeSlots.acquire(node_id, n_gpus)`. On startup the semaphores are reconciled from the DB (rows in `RUNNING/PROFILING` or `QUEUED` with an assigned node pre-acquire their permits). A periodic drift watcher recomputes the authoritative count every five minutes and corrects any divergence.
- **No duplicate containers per row.** Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck `QUEUED` rows cancels any live dispatch task instead of double-releasing its permit.
- **Profiling is sacred.** `Auto-preempt (_preempt_and_release)` refuses to act on `is_profiling_run=TRUE`; `profile-preempt (Phase 1c)` filters victims by `status='RUNNING'` only. Only an explicit user `/stop?reason=user` can cancel an in-flight measurement.

## 2.5 Cross-node checkpoints and image variants

Two GPU generations are supported. The `matemagician` node is pinned to driver 418.x (CUDA 10.1 max), so it runs a separate `:legacy` image built on PyTorch 1.5.1 + Python 3.7. Newer nodes run the default `:latest` image on PyTorch 2.6 + CUDA 12.4. Checkpoints are saved in the legacy pre-1.6 torch serialization format on every node so that a job can resume across nodes regardless of which torch version produced the file.

The shared `data/` directory is mounted on every node via `rclone` over SFTP and contains:

- `data/checkpoints/{job_id}/` — per-job checkpoints, mounted into the container at `/checkpoints`;
- `data/runs/{job_id}/` — captured stdout (`output.log`) and TensorBoard artifacts, mounted at `/runs`;
- `data/shared/data/` — read-only dataset cache (MNIST, CIFAR-10), mounted at `/runs/data`.

Worker-side environment variables `WORKER_GPU_MODE` (`runtime` | `cdi` | `none`) and `IMAGE_TAG_OVERRIDE` let the same `nodes_config` drive heterogeneous nodes: the `polimi-gpu` rootless daemon uses CDI device injection, `matemagician` rewrites `:latest` to `:legacy`, and a CPU-only sandbox can be added by setting `WORKER_GPU_MODE=none`.

## 3 Configuration & Job Submission

This section describes the system configuration files and the job submission API.

### 3.1 Cluster Configuration

Cluster topology is defined in two JSON files loaded at startup (paths configurable via environment variables).

#### 3.1.1 Node Configuration (nodes\_config.json)

Each entry describes a compute node with its GPU resources.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.06,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.55,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

| Field                 | Type          | Description                                                                                                                                                                                                        |
|-----------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id                    | string        | Unique node identifier                                                                                                                                                                                             |
| isForProfiling        | bool          | If true, the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it                                                                                                |
| workerUrl             | string   null | HTTP base URL of the worker on this node, typically reached through an SSH tunnel. When null, dispatch falls through to the in-process JobRunner (only useful for the docker-compose all-in-one development setup) |
| cost                  | float         | Idle cost of the node (\$/h) used by the optimizer                                                                                                                                                                 |
| resources             | array         | GPU resources attached to the node                                                                                                                                                                                 |
| resources[].gpu_type  | string        | GPU model name (e.g. A40, L40S, QuadroP600)                                                                                                                                                                        |
| resources[].gpu_count | int           | Number of GPUs of this type                                                                                                                                                                                        |

Table 1: Node configuration fields.

GPU type and count are declared explicitly in the configuration rather than auto-detected at runtime. This allows mixed-GPU nodes (e.g. a node with both A40 and L40S GPUs), which ANDREAS did not support, and lets the operator declare nodes that aren't physically attached to the API host — the API only ever talks to workers over HTTP.

### 3.1.2 GPU Energy Costs (gpu\_energy\_costs.json)

Maps GPU type and count to hourly energy cost (EUR/h), used by the scheduler.

```
{
  "A40": { "1": 0.15, "2": 0.28, "4": 0.54 },
  "L40S": { "1": 0.18, "2": 0.34, "4": 0.65 }
}
```

## 3.2 Job Submission

**Endpoint:** POST /jobs

### 3.2.1 Request Body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

| Field             | Type     | Required | Description                                                                                               |
|-------------------|----------|----------|-----------------------------------------------------------------------------------------------------------|
| job_id            | string   | yes      | Reusable job type identifier; profiling results are correlated across submissions sharing the same job_id |
| dockerImage       | string   | yes      | Docker image to execute                                                                                   |
| scriptPath        | string   | no       | Path to training script (overrides Docker CMD with python3 <scriptPath>)                                  |
| directoryToMount  | string   | no       | Host directory mounted into the container at /workspace                                                   |
| command           | string[] | no       | Command and arguments (defaults to image entry-point; overridden by scriptPath)                           |
| Priority          | int 1–5  | no       | Job priority (default 3)                                                                                  |
| deadline          | datetime | no       | Job deadline (ISO 8601)                                                                                   |
| batchSize         | int      | no       | Batch size passed as BATCH_SIZE env var                                                                   |
| profilingEpochsNo | int ≥ 1  | no       | Epochs for profiling runs (default 3)                                                                     |
| epochsTotal       | int ≥ 1  | no       | Total training epochs (default 20)                                                                        |

Table 2: Job creation request fields.

Each submission receives a unique UUID instance ID (id) for lifecycle tracking, separate from the job\_id type identifier. When scriptPath is provided, the container runs python3 <scriptPath>; otherwise the Docker image’s default CMD is used. Unlike ANDREAS, Priority and deadline are optional, and an additional command field provides flexibility for non-standard workloads.



### 3.2.2 Response (201 Created)

Returns the full job object including scheduler decisions:

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "node-mixed-prof",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

The response immediately shows the scheduling decision: assigned node, GPU configuration, and whether this is a profiling run.

## 3.3 Full API Overview

Table 3 lists all endpoints. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

| Method | Path                    | Description                       |
|--------|-------------------------|-----------------------------------|
| GET    | /                       | Health check (basic)              |
| GET    | /health                 | Detailed health (DB + job runner) |
| GET    | /nodes                  | List cluster nodes with status    |
| GET    | /gpu-costs              | GPU energy cost table             |
| GET    | /configurations         | Valid GPU configurations          |
| POST   | /jobs                   | Submit a new job                  |
| GET    | /jobs                   | List all jobs (paginated)         |
| GET    | /jobs/{id}              | Get job by ID                     |
| DELETE | /jobs/{id}              | Delete a job                      |
| DELETE | /jobs                   | Delete all jobs                   |
| POST   | /jobs/{id}/stop         | Stop a running job                |
| POST   | /jobs/{id}/resume       | Resume a stopped/failed job       |
| GET    | /jobs/{id}/logs         | Retrieve job output log           |
| GET    | /profiling-results/{id} | Profiling results for a job type  |

Table 3: API endpoints.

All scheduling, profiling, and dispatch decisions are consolidated into a single FastAPI service, in contrast to ANDREAS which split these across a Java Jobs Manager and a C++ Jobs Optimizer. The actual execution lives in per-node *workers* (also FastAPI), which the API reaches over HTTP through SSH tunnels. The external GPUspb optimizer is reached the same way and is responsible for all standard-run placement decisions. IJM exposes explicit `stop`, `resume`, `logs`, and `delete` endpoints, as well as `/profiling-results` and `/configurations` to give users visibility into the scheduler's state.

### 3.4 Runtime environment

The most important runtime knobs (all set in the API process unless noted otherwise) are:

| Variable                     | Default              | Description                                                                                                         |
|------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------|
| DATABASE_URL                 | postgresql://.../ijm | PostgreSQL DSN, reached via SSH tunnel                                                                              |
| OPTIMIZER_URL                | unset                | The API delegates standard-run placement to the GPUspb HTTP service                                                 |
| OPTIMIZER_METHOD             | RG                   | GPUspb solver method passed in the request body                                                                     |
| OPTIMIZER_VERBOSE            | 0                    | GPUspb per-call log verbosity (0–3)                                                                                 |
| PROFILING_CONFIGS_PER_JOB    | 1                    | How many distinct GPU configs a single job <i>instance</i> profiles before falling through to a standard run        |
| EXECUTOR                     | docker               | Executor backend for the in-process JobRunner: docker or mock-slurm (workers always use the docker CLI)             |
| WORKER_GPU_MODE              | runtime              | Worker-side; how Docker exposes GPUs to training containers: runtime / cdi (rootless) / none                        |
| IMAGE_TAG_OVERRIDE           | unset                | Worker-side; rewrites :latest to the given tag. Used on matematician to pick the CUDA-10.1-compatible :legacy image |
| HOST_ROOT, HOST_PROJECT_ROOT | /host, \${PWD}/..    | Resolve container-internal vs. host paths for Docker bind mounts                                                    |

Table 4: Key environment variables.

## 4 End-to-End Scenarios

### 4.1 Objective and cost model

The scheduler minimises

$$\text{TOTALCOST} = \sum_j \text{GPUcost}(j) + \sum_j \text{TardWeight}(j) \cdot \max(0, \text{finish}(j) - \text{deadline}(j)),$$

where GPUcost is wall-clock time times the bundle hourly rate (Table 5, left) and TardWeight grows with job priority (Table 5, right). With slack deadlines the cheapest plan wins; under deadline pressure the tardiness term shifts urgent jobs onto the fastest available bundle.

| GPU bundle   | \$/hour | Priority | TardWeight (\$/sec) |
|--------------|---------|----------|---------------------|
| QuadroP600×1 | 0.030   | 1        | 0.000254            |
| QuadroP600×2 | 0.055   | 4        | 0.000396            |
| A40×1        | 0.300   | 5        | 0.000444            |
| A40×2        | 0.550   |          |                     |

Table 5: Bundle hourly rates and per-priority tardiness weights. Two-GPU bundles include a small co-location discount.

**Cluster.** Two profiling-capable nodes, four slots total:

| Node              | GPUs         | Bundle hourly cost |
|-------------------|--------------|--------------------|
| matemagician (n1) | 2×QuadroP600 | 0.055              |
| polimi-gpu (n2)   | 2×A40        | 0.550              |

For each new job type the profile sweep exercises all four (node, GPU-count) cells (configs\_per\_job=2, four instances).

### 4.2 Available job types

| Type         | Architecture                          | Dataset             |
|--------------|---------------------------------------|---------------------|
| lstm-small   | LSTM 1-layer, 128 hidden              | MNIST (sequence)    |
| lstm-big     | LSTM 3-layer, 256 hidden, dropout 0.2 | MNIST (sequence)    |
| convnet      | 3-layer CNN + BN                      | CIFAR-10            |
| efficientnet | MBConv EfficientNet                   | CIFAR-10            |
| cnn_big      | 10-block CNN, 3→...→512 channels      | synthetic 128×128×3 |

Table 6: Training containers available in runtime/. All share the same checkpoint contract (legacy pre-1.6 torch serialization, loaded with strict=False), so any job can resume across nodes regardless of the destination’s torch version.

| Type         | A40×1 [s/ep] | A40×2 [s/ep] | QuadroP600×1 [s/ep] | QuadroP600×2 [s/ep] |
|--------------|--------------|--------------|---------------------|---------------------|
| lstm-small   | 5.21         | 5.71         | 5.56                | 11.24               |
| cnn_big      | 5.144        | 5.146        | 47.119              | <b>27.904</b>       |
| lstm-big     | —            | —            | —                   | —                   |
| convnet      | —            | —            | —                   | —                   |
| efficientnet | —            | —            | n/a                 | n/a                 |

Table 7: Steady-state per-epoch wall-clock from `profiling_results.duration_seconds` (mean across epochs 2–3, excluding the warmup). Bold cells mark 2-GPU bundles that beat their 1-GPU sibling. `cnn_big` has a substantial 2-GPU win on P600 (1.69×); on A40 the 2-GPU configuration is essentially tied with 1-GPU (5.146 vs 5.144 s/ep), so adding the second A40 doubles the hourly rate for no speed-up and is strictly dominated by A40×1 in the cost model. `efficientnet`’s legacy image uses `torch.nn.SiLU` which is unavailable on the matematician PyTorch 1.5.1 image, so its P600 cells are unmeasurable. Dashes mark types not exercised by the scenarios in this report.

| Type       | A40×1 [\$/ep] | A40×2 [\$/ep] | P600×1 [\$/ep] | P600×2 [\$/ep] |
|------------|---------------|---------------|----------------|----------------|
| lstm-small | 0.000434      | 0.000872      | 0.0000463      | 0.000172       |
| cnn_big    | 0.000429      | 0.000786      | 0.000393       | 0.000426       |

Table 8: GPUcost per epoch = (per-epoch [s]) × (hourly rate from Table 5) / 3600. Multiply by `epochsTotal` to get the optimiser’s energy term for a given plan.

### 4.3 Scenario 1 — single type

We submit five `cnn_big` jobs (50 epochs for the patients, 200 for URGENT) with the priority/deadline mix in Table 9. The mix is the load-bearing detail of the scenario: making all four patients identical (same priority, same deadline) would leave the optimiser with a flat cost surface across “2 jobs on A40, 2 jobs on P600” permutations and force it to break ties between cost-equivalent plans. Staggering priorities (4/4/2/1) and deadlines (tight/tight/slack/slack) gives each job a unique preferred slot, so the resulting placement is reproducible.

| Label   | Type    | Priority | Deadline | Epochs | Submitted at |
|---------|---------|----------|----------|--------|--------------|
| JOB1    | cnn_big | 4        | +15 min  | 50     | $t = 0$ s    |
| JOB2    | cnn_big | 4        | +15 min  | 50     | $t = 5$ s    |
| JOB3    | cnn_big | 2        | +2 h     | 50     | $t = 11$ s   |
| JOB4    | cnn_big | 1        | +2 h     | 50     | $t = 16$ s   |
| URGENT* | cnn_big | 5        | +10 min  | 200    | $t=5.4$ min  |

Table 9: Scenario 1 submissions. Two tight prio-4 patients (the “cheap A40×1 candidates”), two slack low-prio patients (“cheap P600×1 candidates”), one URGENT submitted after the patients have settled into standard runs.

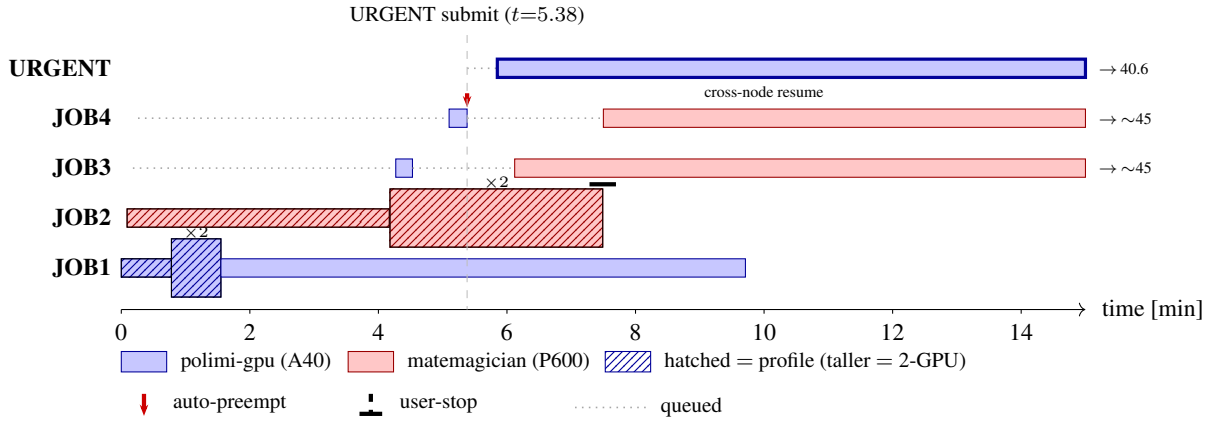


Figure 6: Scenario 1 timeline. *Bar widths are worker container lifetimes*, not measurement durations: profile measurement for a cell completes once the worker has emitted enough post-warmup epoch timestamps (typically within a handful of epochs), but the runtime does not exit on measurement completion — the container keeps training standard epochs until the scheduler either claims a new config (J2’s transition from  $P600 \times 1$  to  $P600 \times 2$  at  $t=4.17$ ) or an external event stops it (URGENT-driven preempt or user-stop). Profile sweep is carried by JOB1 (polimi-gpu, hatched blue:  $A40 \times 1$  then  $A40 \times 2$ ) and JOB2 (matematician, hatched red:  $P600 \times 1$  then  $P600 \times 2 \times 2$ ); together they cover all four cells. `cnn_big` on P600 is  $\sim 8\times$  slower per epoch than on A40, which is why JOB2’s profile bars are much wider than JOB1’s — the per-epoch profile measurements end up being 47 s and 28 s respectively (see Table 10). JOB3 and JOB4 find the cache populated by the time they reach the optimiser and skip the sweep. At  $t=5.38$  min URGENT arrives with a 10-min deadline and 200 epochs — unmeetable on every config, so the optimiser picks the fastest ( $A40 \times 1$ ) and auto-preempts the lowest-priority polimi-gpu resident (JOB4). URGENT dispatches on  $A40 \times 1$  once JOB4’s container has drained ( $t=5.85$ ). At  $t=7.49$  the operator user-stops JOB2 (still in PROFILING — progress 1/50; the user-stop is sticky, JOB2 stays in PREEMPTED), freeing matematician’s P600. JOB4 then *resumes on matematician* from the legacy-format checkpoint — *cross-node* relative to its polimi-gpu origin — and runs to completion on  $P600 \times 1$ . URGENT continues on  $A40 \times 1$  and SUCCEEDS at  $t \approx 40.6$  min (off-chart); JOB3 and JOB4 finish on matematician at  $t \approx 45$  min. Slot use never exceeds 2 jobs/node at any instant.

Table 10 gives the per-bundle measurements that landed in `profiling_results` during the sweep (per-epoch steady-state mean, warmup excluded), along with the 50-epoch and 200-epoch costs the optimiser builds out of them and the bundle hourly rates of Table 5. The `cnn_big` profile has the feature that drives the whole scenario: on polimi-gpu,  $A40 \times 1$  and  $A40 \times 2$  land at *essentially the same* per-epoch time (5.14 vs 5.15 s), so moving from one A40 to two costs another  $\$0.18/h$  for zero speed-up —  $A40 \times 1$  strictly dominates  $A40 \times 2$  on cost any time both options exist.

| Bundle                | per-epoch [s] | GPUcost/ep [\$] | 50-ep cost [\$] | 200-ep cost [\$] | Slack rank  |
|-----------------------|---------------|-----------------|-----------------|------------------|-------------|
| $A40 \times 1$        | 5.144         | 0.000429        | 0.0214          | 0.0857           | 3rd         |
| $A40 \times 2$        | 5.146         | 0.000786        | 0.0393          | 0.1572           | worst       |
| QuadroP600 $\times 1$ | 47.119        | 0.000393        | <b>0.0196</b>   | 0.0785           | <b>best</b> |
| QuadroP600 $\times 2$ | 27.904        | 0.000426        | 0.0213          | 0.0853           | 2nd         |

Table 10: Per-bundle data the optimiser sees for `cnn_big` after S1’s profile sweep. Per-epoch is the steady-state mean from `profiling_results` (warmup epoch excluded); 50-ep / 200-ep costs use Table 5 hourly rates.  $P600 \times 1$  strictly minimises cost for any deadline it can meet; for jobs whose deadline rules out the 39-min  $P600 \times 1$  wall,  $A40 \times 1$  is the cheapest deadline-meeting option.

**Settled S1 placement (post-profile, pre-URGENT).** After the profile sweep, each patient enters its standard run with the optimiser’s per-job choice driven by deadline pressure first, cost second:

- **JOB1, JOB2 (tight, +15 min)** build `optConfig_j` with the deadline-meeting configs only.  $P600 \times 1$ ’s 39-min wall is well past the 15-min budget and so is  $P600 \times 2$ ’s 23-min wall;  $A40 \times 1$  (4.3 min) and  $A40 \times 2$  (4.3 min) both meet it. Of those two,  $A40 \times 1$  is roughly half the price (\$0.0214 vs \$0.0393 for 50 epochs), so the optimiser ranks  $A40 \times 1$  first. JOB1 lands on `polimi-gpu`  $A40 \times 1$  as soon as its profile sweep completes ( $t=1.55$  min) and runs there until SUCCEEDED at  $t=9.71$  min. JOB2, whose own profile sweep was interleaved with JOB3’s slack standard-run dispatch on the second P600 slot, was still in PROFILING at  $t=7.49$  when the operator user-stopped it (final progress 1/50, sticky PREEMPTED).
- **JOB3, JOB4 (slack, +2 h)** face no tardiness on any config, so the optimiser picks the cost-minimum:  $P600 \times 1$ . JOB3 ends up on `matemagician`  $P600 \times 1$  from  $t=6.12$  min; JOB4 transits `polimi` briefly ( $t=5.10$  to  $5.38$  on  $A40 \times 1$ , opportunistically while A40 had a momentarily free slot) before being preempted by URGENT.

The intended steady state at the moment URGENT arrives is therefore {JOB1 on `polimi`  $A40 \times 1$ , JOB4 on `polimi`  $A40 \times 1$ , JOB3 on `matemagician`  $P600 \times 1$ , JOB2 still profiling on `matemagician`} — exactly the placement Figure 6 shows at  $t=5.38$  min.

**URGENT plan comparison (Table 11).** URGENT submits with priority 5, a 10-min deadline, and 200 epochs. Its 200-ep wall is at least 1 029 s on the fastest config, so *every* configuration overruns the 600-s deadline — `optConfig_j` is empty and `optConfig_j_C` is keyed by execution time.  $A40 \times 1$  (1 028.9 s) is fastest by 0.3 s;  $A40 \times 2$  (1 029.2 s) is essentially tied on time but nearly twice the energy cost; P600 configs are an order of magnitude slower. The optimiser picks  $A40 \times 1$ .

| Plan for URGENT                                 | ExecTime [s] | GPUcost [\$] | Tardiness [s] | TardCost [\$] | TotalCost [\$] | Decision |
|-------------------------------------------------|--------------|--------------|---------------|---------------|----------------|----------|
| $A40 \times 1$ (preempt JOB4)                   | 1 029        | 0.0857       | 429           | 0.1905        | <b>0.2762</b>  | ✓ chosen |
| $A40 \times 2$ (preempt JOB1+JOB4)              | 1 029        | 0.1572       | 429           | 0.1906        | 0.3478         | rejected |
| $P600 \times 2$ (preempt JOB3, profile pending) | 5 581        | 0.0853       | 4 981         | 2.2116        | 2.2969         | rejected |
| $P600 \times 1$ (preempt JOB3)                  | 9 424        | 0.0785       | 8 824         | 3.9178        | 3.9963         | rejected |

Table 11: Optimiser plan comparison for URGENT. `ExecTime` =  $200 \times$  per-epoch from Table 10; tardiness measured against URGENT’s 10-min deadline at submission; `TardCost` uses the priority-5 weight  $w_5=0.000444$  \$/s (Table 5).  $A40 \times 1$  wins because its time is essentially tied with  $A40 \times 2$  but the energy cost is roughly half, and it only needs to evict *one* `polimi` resident (the lowest-priority one, JOB4) instead of both.  $P600 \times 2$  sits at second cheapest on paper but its profile was still incomplete at submission time, so the heuristic could not commit to it.

**Why JOB4 specifically.**  $A40 \times 1$  needs one A40 GPU. At URGENT’s submission both A40 slots on `polimi-gpu` are taken (JOB1 and JOB4). The optimiser drops the candidate from `currentScheduling` that produces the lowest residual cost — evicting JOB4 costs less than evicting JOB1 because JOB4 has priority 1 (TardWeight 0.000254 \$/s) against JOB1’s priority 4 (TardWeight 0.000396 \$/s), and JOB4’s displacement to P600 incurs less tardiness penalty per second than JOB1’s would.

**Cross-node resume (Stage 4).** JOB4 is queued at  $t=5.38$  with `assigned_node=NULL` and a checkpoint at the last epoch boundary on disk. `Polimi-gpu` is fully booked by URGENT and JOB1, and `matemagician`’s free P600 slot is held by JOB3 (slack standard) while the other is held by JOB2 (still profiling). At  $t=7.49$  the operator user-stops JOB2; `matemagician` P600 frees one slot; the next scheduler round dispatches JOB4 to `matemagician`  $P600 \times 1$ . JOB4 loads the checkpoint emitted on `polimi-gpu` (legacy pre-1.6 torch serialisation format), and resumes from epoch 1 — *cross-node* relative to

its polimi-gpu origin. This exercises the checkpoint-format compatibility path that lets jobs migrate between the modern PyTorch 2.6 image on polimi-gpu and the PyTorch 1.5.1 legacy image on matemagician (whose NVIDIA driver maxes out at CUDA 10.1).

#### 4.4 Scenario 2 — two types, forced 2-GPU placement

| Label    | Type           | Priority | Deadline | Epochs |
|----------|----------------|----------|----------|--------|
| PIN-A    | B (cnn_big)    | 5        | +10 min  | 80     |
| PIN-B    | B (cnn_big)    | 5        | +10 min  | 80     |
| P-LSTM-1 | S (lstm-small) | 1        | +8 h     | 200    |
| P-LSTM-2 | S (lstm-small) | 1        | +8 h     | 200    |
| URGENT*  | B (cnn_big)    | 5        | +22 min  | 40     |

Table 12: Scenario 2 submissions. PIN-A/B are tight-deadline `cnn_big` jobs that the optimiser is forced to place on polimi-gpu A40 $\times$ 1 (P600 $\times$ 1 would be 68 min runtime vs their 10-min deadline). P-LSTM-1/2 are slack lstm-small jobs that land on matemagician P600 $\times$ 1. URGENT arrives once steady state holds; its tight deadline admits P600 $\times$ 2 (20 min) but not P600 $\times$ 1 (34 min), and the A40 plans are blocked by the priority-5 tight-deadline PIN victims.

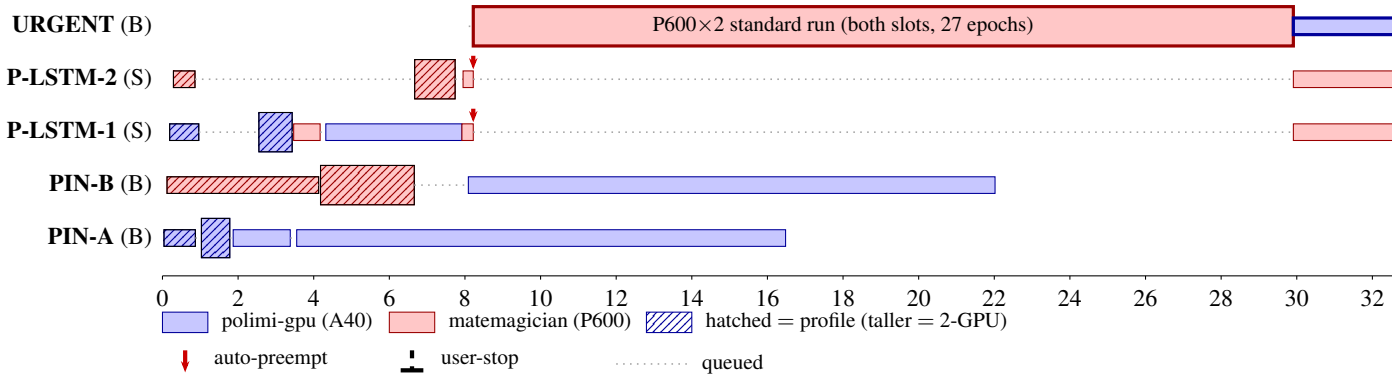


Figure 7: Scenario 2 timeline (snapshot ground truth, 0.5 s poll). Profile sweep covers all 8 cells across two types. PIN-B’s huge hatched bar (0.12–4.13 min) is cold `cnn_big` P600 $\times$ 1 profile: 47.7 s/epoch  $\times$  3 epochs + container start. Steady state at  $\sim$ 8 min: both A40 slots hold the `cnn_big` pins, both P600 slots hold the lstm-small patients. At  $t=8.22$  URGENT arrives and the optimiser places it on *matemagician*  $\times$  *QuadroP600* $\times$ 2 (taller red bar), evicting *both* lstm-small patients in a single cascade — the central result. URGENT runs 27 epochs on P600 $\times$ 2 from 8.22 to 29.91, then migrates to polimi A40 $\times$ 1 (freed when PIN-A and PIN-B finished at 16.48 and 22.02) and completes the final 13 epochs there (SUCCEEDED 32.65). At 29.91, both lstm-small patients resume on the freed P600 slots from their checkpoints and finish  $\sim t=60$  min (off-chart). Slot use never exceeds 2 jobs/node at any instant.

**Settled steady state (before URGENT).** PIN-A, PIN-B (`cnn_big`)  $\rightarrow$  polimi-gpu A40 $\times$ 1 (forced: `cnn_big` on P600 $\times$ 1 takes 68 min vs their 10-min deadline). P-LSTM-1, P-LSTM-2 (`lstm-small`)  $\rightarrow$  matemagician P600 $\times$ 1 (cheapest for slack lstm-small; A40 is closed by the priority-5 pins).

**URGENT plan comparison.** URGENT arrives at  $t=8.10$  min with deadline +22 min.



| Plan                                    | ExecTime [s] | GPUcost [\$] | Tardiness [s] | TardCost [\$] | Total [\$]             | Decision |
|-----------------------------------------|--------------|--------------|---------------|---------------|------------------------|----------|
| A40×1 (preempt 1 pin)                   | 252          | 0.0210       | 0             | 0.0           | $\approx 1.57^\dagger$ | rejected |
| A40×2 (preempt both pins)               | 225          | 0.0344       | 0             | 0.0           | $\approx 3.12^\dagger$ | rejected |
| P600×1 (preempt 1 lstm-small)           | 2042         | 0.0170       | 722           | 0.320         | 0.337                  | rejected |
| <b>P600×2 (preempt both lstm-small)</b> | <b>1213</b>  | 0.0185       | 0             | 0.0           | <b>0.019</b>           | ✓ chosen |

Table 13: Optimiser plan comparison for URGENT. ExecTime is  $40 \times$  per-epoch from Table 7; tardiness measured against URGENT’s deadline (submit +22 min); TardCost uses the priority-5 weight from Table 5.  $^\dagger$  A40 plans appear free of URGENT’s own tardiness, but their *cascade cost* dominates: each priority-5 pin victim re-dispatched onto P600×1 becomes 58 min tardy on its own 10-min deadline ( $\approx \$1.55/\text{victim}$ ). P600×2 evicts only the slack lstm-small (+8 h deadline) so its cascade is  $\approx \$0$ . Captured run confirms the chosen plan.

**Measured cascade ( $t=8.22$  min).** URGENT (92f64b0c) was placed on *matemagician* × *QuadroP600*×2 in  $\sim 12$  s after submission. Both patient lstm-small jobs (d2b87e00, 44e54d48) flipped to QUEUED with cleared `assigned_node` in the same optimiser round — a single auto-preempt cascade with two simultaneous victims. URGENT ran on P600×2 for 21.7 min (27/40 epochs), then once PIN-A and PIN-B vacated polimi A40 (SUCCEEDED at 16.48 and 22.02), the optimiser migrated URGENT to polimi A40×1 at  $t=29.91$  min where it finished the remaining 13 epochs in 2.7 min (SUCCEEDED 32.65). At the same moment, both lstm-small patients resumed on the freed P600 slots from their checkpoints and trained on through  $t\approx 60$  min. PIN-A/B were untouched throughout: their priority-5 + tight-deadline status priced any A40 eviction cascade out of the optimiser’s reach.

## 4.5 Reproducibility

Per-bundle costs in Tables 7, 8, 10 and ?? are taken from `profiling_results.duration_seconds` (the steady-state per-epoch mean produced by `compute_duration` in `worker/profiling.py`, excluding the warmup epoch) multiplied by the hourly rates in Table 5. Timeline figures mirror per-instance state transitions captured by a 0.5 s API poller; each bar segment in Figures 6 and 7 corresponds to a contiguous run of identical (`status`, `assigned_node`, `assigned_gpu_config`, `is_profiling_run`). The two scenarios are driven by `infra/e2e_scenario.sh` (single-type, profile sweep + urgent + user-stop + cross-node resume) and `infra/e2e_scenario_2types.sh` (two-type, single urgent P600×2 preempt cascade). Both scripts print one line per assertion and expose the slot tracker via `GET /admin/slots`; the API logs every scheduler/optimiser round to `/tmp/api.log` when launched with `infra/launch.sh tunnel`.



## References

- [1] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.